2015 IEEE/ACM 37th IEEE International Conference on Software Engineering

Evolution of Software Development Strategies

Katrina Falkner, Claudia Szabo, Rebecca Vivian and Nickolas Falkner
School of Computer Science
The University of Adelaide,
Adelaide, Australia
Email: firstname.lastname@adelaide.edu.au

***Abstract*—The development of discipline-specific cognitive and meta-cognitive skills is fundamental to the successful mastery of software development skills and processes. This development happens over time and is influenced by many factors, however its understanding by teachers is crucial in order to develop activities and materials to transform students from novice to expert software engineers. In this paper, we analyse the evolution of learning strategies of novice, first year students, to expert, final year students. We analyse reflections on software development processes from students in an introductory software development course, and compare them to those of final year students, in a distributed systems development course. Our study shows that computer science - specific strategies evolve as expected, with the majority of final year students including design before coding in their software development process, but that several areas still require scaffolding activities to assist in learning development.**

I. Introduction

The development of deep learning strategies, self-regulation, abstract thinking and metacognitive strategies are vital in order to assist students in achieving success [1], [2]. A student with self-regulated learning behaviours will set their goals, determine and allocate their resources, as well as manage their time effectively [3]. Without this fundamental level of metacognition, students cannot direct their knowledge in a useful and constructive manner and thus are unlikely to succeed. A significant aspect in the development of self-regulating learning (SRL) strategies is the ability to monitor and reflect upon those strategies within the context of Computer Science (CS) as a discipline, enabling the individual to identify their success or failure, identify strategies to apply in specific contexts, and develop new strategies [4], [5]. Allwood [6] identifies that novices tend to use more general strategies rather than the more powerful specialised strategies employed by experts. According to Robillard [7], expert programmers tend to adopt a systematic planning process, based upon access to the conceptual knowledge required to complete the specified task, enabling a breadth-first search of the problem space. Novice programmers, however, build their planning and design processes upon their knowledge of programming languages, resulting in a depth-first search and a focus on concrete rather than abstract argumentation.

The transition from novice to expert is assisted by reflection on prior successes and failures [8], followed by analysis of potential areas for improvement. Before we can assist our students in the process of reflection and self-regulation, we must identify and articulate successful SRL strategies for the CS context [9]. Therefore, we must develop an understanding of those discipline specific strategies that can be successfully learnt and adopted by students [10].

In our previous work [11], we analysed students' reflections on their SRL processes as applied to introductory software development. Using a grounded theory model of qualitative analysis, we were able to identify SRL strategies that are specific to software development, expressed in the students' own words and relative to their own experiences. We presented a detailed analysis of the nature of these discipline-specific SRL strategies and how these strategies contribute to the learning of novice students. In this paper, we explore the evolution of discipline-specific SRL strategies through the combined analysis of a cohort of novice students, and a second cohort of final year students. We present an analysis of the evolution of SRL strategies from novice to expert learners, with the aim of identifying points where the development and application of identified SRL can be improved. The analysis of this evolution is fundamental for the development of targeted scaffolding and support to address identified issues. Our findings show that students develop a range of sophisticated strategies, that can be readily scaffolded within the curriculum.

II. Related Work

Self-regulated learners have been defined by Zimmerman [2] as those that "plan, set goals, organise, self-monitor and self-evaluate". The development of SRL strategies has been found to be a complex issue, associated with the perceived purpose of engagement with the activity, the students self-perception of their ability, and the situated context of the activity - these three factors impact upon the self-regulation strategies that the student then considers relevant for application [12]. Lichtinger and Kaplan [13] call for the identification of domain and context-specific purposes of engagement, and the articulation of types of SRL strategies that would be desirable for students within that domain. Further, the development of effective domain-specific SRL strategies, such as domain-based design and planning, can assist in the application and development of other SRL strategies [14]. Kramer [15] describes the difference between novice and expert software engineering students as their 'ability to perform abstract thinking and to exhibit abstraction skills'.

The categorisation of 'expert-novice' [16], a subset of novices who are able to progress quickly in their development of discipline knowledge, presents us with an interesting

978-1-4799-1934-5/15 $31.00 © 2015 IEEE
DOI 10.1109/ICSE.2015.153

ICSE 2015, Florence, Italy
Joint SE Education and Training

challenge. Expert-novices are able to progress at a more rapid pace through a combination of intellectual development and advanced metacognitive strategies, however, it is not known which of these is the predominant factor, and whether the isolated development of metacognition, in our case, software development processes, in novices will assist in their assimilation of discipline knowledge.

Few existing works explicitly address SRL strategies applicable to the software development process itself, and most tend to focus on novice programmers. Novices in any area lack the detailed, well-organised domain knowledge that is necessary to produce effective problem solving processes - this may be seen in software development as opportunistic and arbitrary design and implementation [7]. Novice problem solvers struggle to understand conceptually difficult problems, due in part to their lack of deep discipline knowledge, and tend to approach the problem using known techniques from their knowledge of programming languages, and a process of impasses and local repairs [14]. As they are unskilled in planning, novices also tend to delay planning relevant to the problem solving process itself until it is absolutely necessary, imposing further delays in their development of that skill set.

Bergin et al. [17] explore the relationship between SRL and introductory programming performance, specifically the use of metacognitive strategies, finding that students who perform well in programming tasks also exhibit frequent use of metacognitive behaviours. Violet [18] explores the cooperative development of metacognitive strategies for an introductory computer science course, involving students in both the development of the strategy and the identification of modelling and coaching procedures designed to guide students through the strategy. The strategy introduced builds upon theories of social constructivism and cognitive development [19], [20], and follows a simple five-step process of software design. They found that this instructional approach resulted in significant changes in students' cognitive learning outcomes. Caruso et al. [21] explore a grounded theory analysis of students' reflections on their software development process, identifying a focus on non-discipline aspects of self-regulation and an inability to develop appropriate plans for improving their SRL strategies. Hanks et al. [9] identify similar results in a "saying is believing" experiment where students completing a course on introductory programming were asked to provide advice for the next cohort. They report a combination of general advice (63%), attitudinal advice (34%) and programming-specific advice (23%), identifying a relative dearth of work on discipline-specific strategy.

A large body of work exists in methods for teaching software engineering process models, either focusing on using traditional lecture-based formats, team projects [22], or games [23]; the Personal Software Process [24] is taught in a variety of curricula, with various degrees of success [25], [26]. However, few works focus on understanding how software engineering knowledge is built by learners. Bareiss et al. [27] analysed the evolution of the software engineering knowledge of 16 master students from the beginning of their study until the end. The study found that the students' mental models of requirements analysis and engineering management improved significantly towards the end of their degree, but concluded that richer and more in-depth analyses of how students develop their understanding and software engineering process is needed. In our previous work [11], we identified CS-specific self-regulated learning behaviours as novice students applied them to introductory software development. We distinguished between successful and unsuccessful strategies, which included the use of design and an aid to software development, the process of assessing difficulty, as well as time management and software decomposition strategies.

III. Methodology

In this paper, we undertake a quantitative and qualitative analysis of student reflections on their software development processes and SRL strategies in order to identify those strategies that are specific to CS, and how they evolve from the experiences of novices to those of a final year student cohort. We explore the nature of this evolution, and consequently what approaches to scaffold and support such evolution are needed. Our research questions are:

- *What CS-specific SRL strategies do final-year students articulate as using in programming assignments?*
- *What are the differences between the SRL strategies as identified by final-year students and those identified by first-year students?*

A. Research Method

Case studies are a suitable method for examining interactions and behaviours between individuals [28]. An instrumental case study approach, one that describes a typical case relevant to the research topic, uses a particular case as an illustration, in our case to understand and elaborate Students' SRL strategies within software development [28], [29]. Case studies capture the complexities of a phenomenon; such detailed observations cannot be captured in surveys or experimental designs [30]. The number of participants in our study is 85 for the novice cohort, and 38 for the final year student cohort - this number of participants is sufficient, with samples sizes for qualitative research ranging from 1 to 99, with an average sample size of 22.

This project has adopted a mixed-method case study design where both quantitative and qualitative data were collected. The data were subjected to grounded theory analysis, starting with a process of open coding, before proceeding to axial coding using the coding framework established in our previous work [11]. Since the focus is both on establishing the SRL behaviours of more experienced students, but also to analyse the evolution from first year to third year SRLs, we decided to employ the same coding framework as before and use grounded theory for content that did not fit.

Grounded theory involves the establishment of a coding framework and analysis environment derived from the data itself. Grounded theory differs from other types of qualitative analysis in that a specific, structured coding framework is not

employed. The first stage in grounded theory development is open coding, where the data is broken down into distinct segments in order to obtain the full collection of ideas and concepts present in the data, without regard to how it will be used. Subsequently, axial coding is employed, where the coding framework developed during the open coding stage is refined and reorganised into specific categories, informed by theoretical frameworks and comparison within the data. There are significant advantages in the adoption of a grounded theory approach, in contrast to directed content analysis with an established coding framework, including removing the potential to force fit observations into existing categories and misclassification.

The case studies considered in the research project consist of an introductory programming course, and a final year distributed systems course at the University of Adelaide. The first case study considers a final course in an introductory programming sequence. This aim of this course is for students to build upon their existing programming fundamentals to introduce simple data structures, and the algorithms that proceed from them, and software engineering principles. Students within this course have completed 1-2 prior programming courses, based upon their incoming experience levels. These introductory programming courses have focussed on the knowledge and application of fundamental programming constructs within small scale problem solving. As such, this course contains, for many students, their first experiences with non-trivial complexity software design and development. Initial results from this case study are described in [11].

The second case study, the final year distributed systems course, explores the challenges faced in developing distributed systems, including failures, distributed time, and multiple address spaces, and the range of techniques introduced to address these challenges, such as agreement protocols, consistency management, and distributed transactions. The course contains a significant application context, where students are required to design and implement distributed systems that explore both challenges and mitigation techniques. Students within this course have taken an introductory software engineering course, and in general are taking our capstone software engineering course at the same time with the distributed systems course. In both courses discussed in this paper, students are requested to complete a substantial reflective exercise that requires students to describe their current software development processes, how they have changed and a description of how they intend to change them in the future.

B. Coding Framework

The basic unit of analysis in this project were coding units [31], including sections of text responses, of any size. Within the open coding stage, sections of text, such as a sentence, word or phrase, were coded while the selection represented a single idea or concept related to SRL strategy. In excess of 361 pages from 123 student reflections were coded using the qualitative analysis software NVivo (version 10) defining an initial set of 78 distinct codes. The research team methodically worked through the student reflections, coding their observations either to existing nodes within the framework or to a newly created node, identifying a description of the newly created node and exemplar. During the axial coding stage, the researcher worked in collaboration with the project team to establish inter-rater reliability, and to iteratively refine the established codes into categories, merging codes where appropriate, informed by existing coding categories related to SRL as defined by Zimmerman [32] and identifying discipline-specific categories as derived from the data (see Table I).

The content analysis frequencies, combined with qualitative examples of student descriptions of their behaviours and strategies, are discussed further below.

IV. Quantitative Analysis

Our analysis reveals that students utilise a range of SRL strategies, including general strategies, Computer Science specific strategies which are adapted and articulated within the context of CS, and strategies that are specific and newly introduced to address the learning challenges of CS. Further, we are able to categorise these strategies as (a) those that were perceived to lead to success, and (b) those that led to failure.

A. Case 1: Novices

Tables I and II present the identified strategies associated with successful behaviour, classified as either CS-specific or general strategies. Table I identifies a clear focus on software design processes, with a dominant use of diagrams to help describe or explain their software design, and the development of sub-goals based on their software designs (and subsequent software decomposition). Prioritisation is also indicated as a key strategy - in this case referring to the prioritisation of design activity over subsequent implementation and testing activities. Assessing problem difficulty is a crucial step in planning and goal-setting, with several CS-specific strategies identified within this category. Table II reports the identified general SRL strategies; these were frequently articulated within the CS context, with contextual description added to illustrate how the strategy was employed or modified within the domain.

Students also indicated strategies that led to unsuccessful behaviours. The majority of these strategies are general strategies, and are frequently in direct opposition to a contrasting strategy found to be successful. Table III presents those general strategies found to be unsuccessful by students, focusing primarily on poor time management, inadequate goal setting and insufficient planning strategies. Interestingly, students identified poor strategies associated with focusing their activity based on the assessment, i.e. directing their effort based on how many marks individual parts of the assignment were worth. As a smaller fraction of the marks was associated with design than implementation functionality, as a consequence, these students did not place a high priority on undertaking design activities, and typically developed a design document subsequent to the completion and testing of their

TABLE I: CS-specific SRL strategies and their indicated frequency (total count = 448).

Strategy	Freq	% Freq
Development Process	*255*	*56.9%*
Use diagrams to describe or explain design	119	26.6%
Use design to understand problem or code	45	10.0%
Develop design before coding	22	4.9%
Document design or code for future use	20	4.5%
Validate design	18	4.0%
Comprehensive test cases	15	3.3%
Develop design concurrently with coding	9	2.0%
Develop design after coding	4	0.9%
Use design principles or standards	3	0.7%
Time Management	*68*	*15.2%*
Prioritisation	59	13.2%
Design as aid to time management	8	1.8%
Allocate time for prototyping	1	0.2%
Decompose problem	*61*	*13.6%*
Use decomposition to develop testing strategy	41	9.2%
Create plan from design	20	4.5%
Assess Difficulty	*54*	*12.1%*
Algorithm complexity	23	5.1%
Prototyping and experimentation	12	2.7%
Use design to assess difficulty	10	2.2%
Number of concepts, stages or components	7	1.6%
Need for design driven by complexity	2	0.4%
Build Knowledge	*10*	*2.2%*
Practice writing code	10	2.2%

code. Table IV describes the CS-specific strategies associated with failure, reflecting this observation.

B. Case 2: Final Year Cohort

Using the coding framework developed from our exploration of novice strategies, an analysis was undertaken of the final year cohort reflections, identifying the frequency of codes already identified as well as additional codes specific to this case study. Tables V and VI show the identified successful strategies, classified as either CS-specific or general strategies, as defined by Zimmerman [32].

While a comparison with those successful strategies identified by the novice cohort identifies similarities in the focus on software development strategies, a clear difference is visible in the range of popular strategies identified by the final year cohort, including an increased reliance on design principles or standards, and introducing new strategies such as refinement of design, identifying and prioritising core components in the design, and a focus on integration requirements. Although only 4.69% of comments related to prototyping and experimentation as a successful strategy, further analysis reveals that approximately 24% of the final year cohort identified that as a successful strategy, in comparison to approximately 10% of novice students.

Overall, the final year cohort identified a significant greater focus on CS-specific SRL strategies in their reflections, with a ratio of CS to general strategy of 1.6:1, in comparison with the novice cohort, where strategies were equally identified across both groups. Where the final year cohort did identify general SRL strategies that led to success, they were broadly similar in their use of strategies designed to assist in assessing the

TABLE II: General SRL strategies and their indicated frequency (total count = 455).

Strategy	*Freq*	*%Freq*
Assess Difficulty	*127*	*27.9%*
Identify new knowledge that is needed	42	9.2%
Assess own ability	24	5.3%
Assess difficulty - no specific strategy	23	5.1%
Understand question	20	4.4%
Compare with previous	12	2.6%
Specification – length, assessment	5	1.1%
Ask friends	1	0.2%
Time Management	*105*	*23.1%*
Fixed timetable - sufficient	59	13.0%
Time Estimation	26	5.7%
Overestimate time	16	3.5%
Time management - no specific strategy	4	0.9%
Build Knowledge	*81*	*17.8%*
Talk to friends or lecturers	42	9.2%
Access resources	39	8.6%
Decompose problem	*72*	*15.8%*
Create plan from specification	58	12.7%
Decompose - no specific strategy	12	2.6%
Frequent accomplishment	2	0.4%
Personal Management	*70*	*15.4%*
Reflection and changing stratagies	49	10.8%
Self assessment	10	2.2%
Avoid sources of anxiety	8	1.8%
Avoid working late	2	0.4%
Reduce distractions	1	0.2%

TABLE III: General unsuccessful SRL strategies and their indicated frequency (total count = 222).

Strategy	*Freq*	*%Freq*
Time Management	*115*	*51.8%*
Poor prioritisation of competing demands	42	18.9%
Underestimated time	33	14.9%
Poor time management	14	6.3%
Fixed timetable - insufficient	11	5.0%
Procrastination	5	2.3%
Time management - no specific strategy	5	2.3%
Avoiding difficult tasks	4	1.8%
Assume nothing will go wrong	1	0.5%
Assess Difficulty	*57*	*25.7%*
Underestimated complexity	35	15.8%
Lack of fundamental skills	13	5.9%
Misunderstand requirements	7	3.2%
Did not assess difficulty	2	0.9%
Personal Management	*33*	*14.9%*
Assessment achievement focus	31	14.0%
No reflection leading to change	2	0.9%
Build Knowledge	*10*	*4.5%*
Did not seek help or ignored help	8	3.6%
Not attending class	2	0.9%
Decompose problem	*7*	*3.2%*
Did not decompose	7	3.2%

TABLE IV: CS-specific unsuccessful SRL strategies and their indicated frequency (total count = 70).

Strategy	*Freq*	*%Freq*
Development Process	*70*	*100.0%*
Coding before Design	40	57.1%
Incomplete design	29	41.4%
Insufficient testing	1	1.4%

TABLE V: Final year cohort CS-specific SRL strategies and their indicated frequency (total count = 256).

Strategy	Freq	% Freq
Development Process	*165*	*64.45%*
Develop design before coding	33	12.89%
Use design principles or standards	29	11.33%
Use diagrams to describe or explain design	25	9.77%
Comprehensive test cases	22	8.59%
Refine, refactor code or design	17	6.64%
Use design to understand problem or code	9	3.52%
Core component development	7	2.73%
Document design or code for future use	7	2.73%
Integration focus	6	2.34%
Develop design concurrently with coding	5	1.95%
Validate design	5	1.95%
Assess Difficulty	*33*	*12.89%*
Prototyping and experimentation	12	4.69%
Use design to assess difficulty	8	3.13%
Number of concepts, stages or components	6	2.34%
Need for design driven by complexity	4	1.56%
Algorithm Complexity	3	1.17%
Time Management	*30*	*11.72%*
Prioritisation	16	6.25%
Design as aid to time management	11	4.30%
Allocate time for prototyping	3	1.17%
Decompose Problem	*27*	*10.55%*
Create plan from design	11	4.30%
Decomposition driven by quality attributes	9	3.52%
Use decomposition to develop a testing strategy	5	1.95%
Seek out requirements for planning	2	0.78%
Build Knowledge	*1*	*0.39%*
Practice writing code	1	0.39%

TABLE VI: Final year cohort general SRL strategies and their indicated frequency (total count = 161).

Strategy	Freq	%Freq
Assess Difficulty	*60*	*37.27%*
Understand question	16	9.94%
Assess own ability	12	7.45%
Identify new knowledge that is needed	11	6.83%
Compare with previous	8	4.97%
Specification - length, assessment	7	4.35%
Ask friends	3	1.86%
No specific strategy	3	1.86%
Personal management	*44*	*27.33%*
Reflection and changing strategies	36	22.36%
Self assessment	6	3.73%
Avoid sources of anxiety	1	0.62%
Avoid working late	1	0.62%
Reduce distractions	0	0.00%
Decompose Problem	*21*	*13.04%*
Create plan from specification	9	5.59%
Decompose - no specific strategy	6	3.73%
Frequent accomplishment	6	3.73%
Build Knowledge	*19*	*11.80%*
Talk to friends of lecturers	10	6.21%
Access resources	9	5.59%
Time management	*17*	*10.56%*
Time estimation	7	4.35%
Fixed timetable - sufficient	6	3.73%
No specific strategy	3	1.86%
Overestimate time	1	0.62%

difficulty of their assignment work, but with a stronger focus on personal management, and a decrease in dependence on general time management strategies. A significant focus on reflection as an aid to improvement of their processes and strategies is identified with 22.36% of comments relating to this strategy.

We show the evolution of CS-specific strategies for software development process from novice to final year students in Fig. 1 (a) and Fig. 1(b) respectively, focussing on design strategies and aggregating the other comments under ”Other” in the legend. As it can be seen, only 1%of the novice comments with respect to development process focus on the use of standards; in contrast, 18% of the comments of final year students focus on this. In a similar manner, there is a drastic increase in the percentage of comments focused on developing design from novice to final year students, suggesting that the use of designs in software development has become fairly mainstream.

In contrast, when identifying strategies that led to unsuccessful behaviours, the final year cohort focused primarily on unsuccessful time management strategies, clearly indicating that time management issues are still a major factor contributing to lack of success. Interestingly, difficulties in seeking or accessing help did not appear as a factor for the final year cohort. When analysing the CS-specific SRL strategies that were deemed to be unsuccessful, the final year cohort placed a greater emphasis on incomplete design processes, with only one comment identifying the development of design subsequent to implementation.

V. Qualitative Analysis

Our quantitative analysis has shown that final year students have more sophisticated software development processes than novice students. Final year students consistently use design as a means to create a solution but also to understand a problem. Their software development process is iterative, and they use standards and established software engineering terminology to discuss their solutions. Similarly, the way students assess task difficulty improves over years. We discuss these themes in the following.

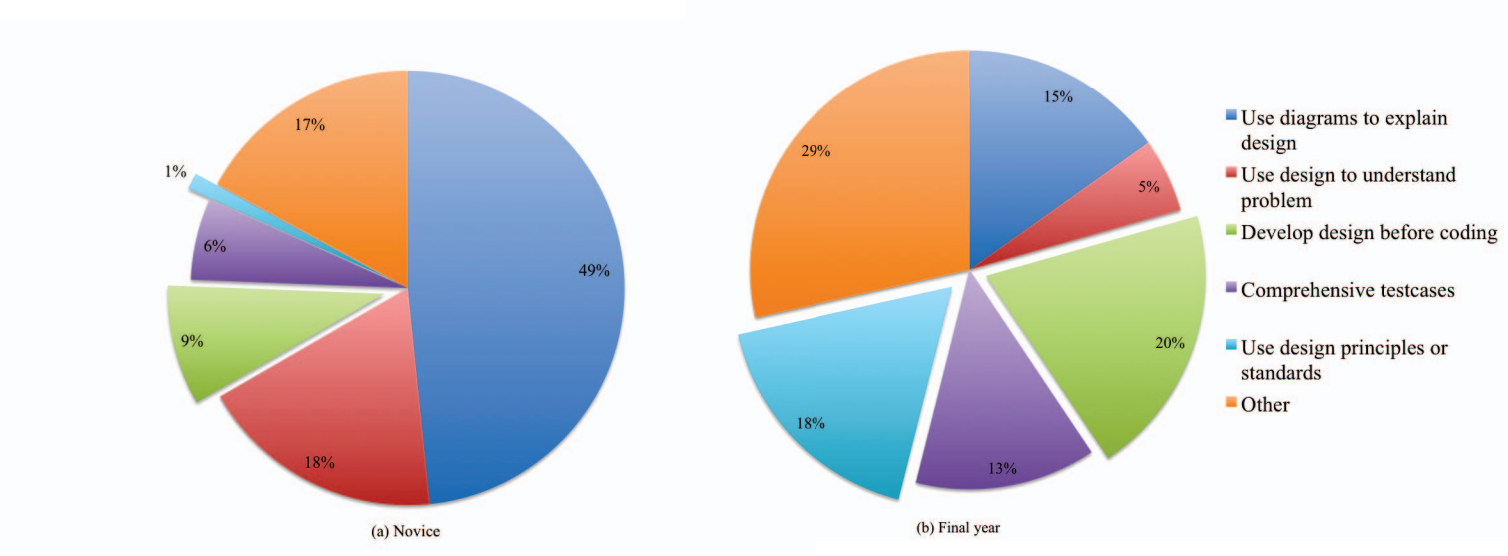

Fig. 1: CS-specific successful strategies in software development process.

TABLE VII: Final year cohort general unsuccessful SRL strategies and their indicated frequency (total count = 45).

Category	Freq	% Freq
Time management	*32*	*71.1%*
No specific strategy	12	26.7%
Underestimated time	7	15.6%
Procrastination	5	11.1%
Poor prioritisation of competing demands	3	6.7%
Poor time management	3	6.7%
Avoiding difficult tasks	2	4.4%
Assume nothing will go wrong	0	0.0%
Fixed timetable - insufficient	0	0.0%
Assess Difficulty	*9*	*20.0%*
Underestimated complexity	4	8.9%
Did not assess difficulty	3	6.7%
Lack of fundamental skills	1	2.2%
Misunderstood requirements	1	2.2%
Personal management	*3*	*6.7%*
Assessment achievement focus	2	4.4%
No reflection leading to change	1	2.2%
Decompose Problem	*1*	*2.2%*
Did not decompose	1	2.2%
Build Knowledge	*0*	*0.0%*
Did not seek help or ignored help	0	0.0%
Not attending class	0	0.0%

TABLE VIII: Final year cohort CS-specific unsuccessful SRL strategies and their indicated frequency (total count = 17).

Strategy	Freq	% Freq
Development Process	*17*	*100.00%*
Incomplete design	11	64.71%
Do not follow development plan	3	17.65%
Insufficient testing	2	11.76%
Develop design after coding	1	5.88%
Develop design concurrently with coding	0	0.00%

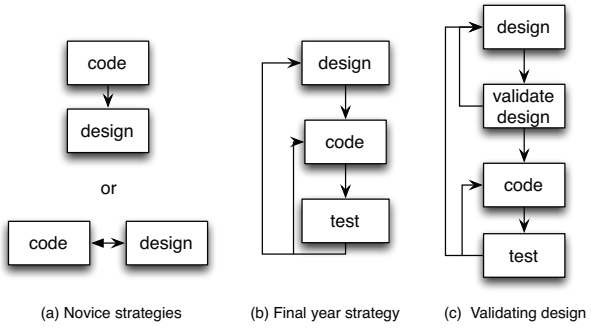

Fig. 2: Spectrum of design as strategy across cohorts.

A. *Design as Strategy*

The spectrum of using design as strategy varies across cohorts, as shown in Fig. 2.

In contrast to novices who tend to design after coding or at best design in parallel with coding, final year students are more mature in their approach. The strategy employed by a final year student is in general that of designing a solution, coding it, testing the code, and, if necessary, either refining or refactoring the code or the design. Despite the significant reduction in development time that the validation of design before coding brings, few students report using it. A student comments:

"usually I'll draw diagrams and execute my algorithm by hand to verify that it's going to work as I'd expect. "

Fig. 3 show a process diagram drawn by a final year student, with an emphasis on validating and testing the design before implementation.

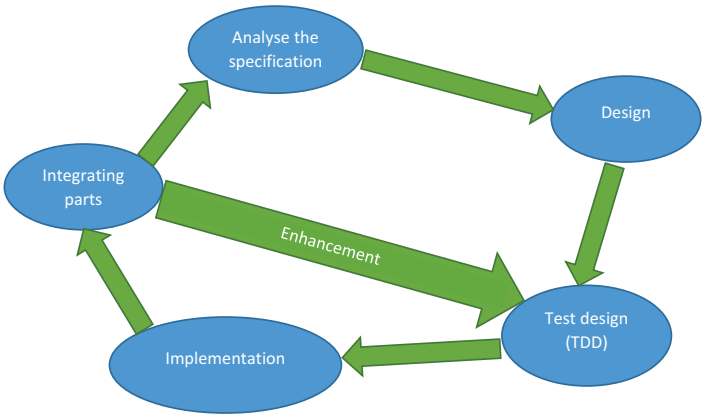

Fig. 3: Process diagram - validate design.

The process of designing, validating design, and testing is also iterative:

"As the designs are not always set in stone, many revisions are often made with consideration of the software specification, use cases, test cases, areas of concern with reference to the specification and the areas of the program that would need to be tested thoroughly."

B. Iterative Processes

Few novice students report following iterative or incremental processes in implementing their assignments, in contrast to final year students, where these processes are common. A significant difference between novices and final year students is with respect to understanding and discussing these processes, in that final year students have the knowledge and terminology to discuss them. Moreover, final year students will heavily employ standards and software engineering process models in implementing their assignments. Fig. 4 and 5 show the stark differences between a process diagram drawn by a novice student, and that drawn by a final year student.

C. Decomposition

Another aspect that differentiates final year students from novices is the focus on software quality attributes such as loose coupling, cohesiveness or repeated behaviour in decomposition and design. One student acknowledges that duplicated functionality is not a desired quality attribute:

"I often find shared behaviour which may call for a change of design to include private helper methods, abstract classes, or even a complete change of structure."

Another student notes the importance of loose coupling and cohesion for maintenance:

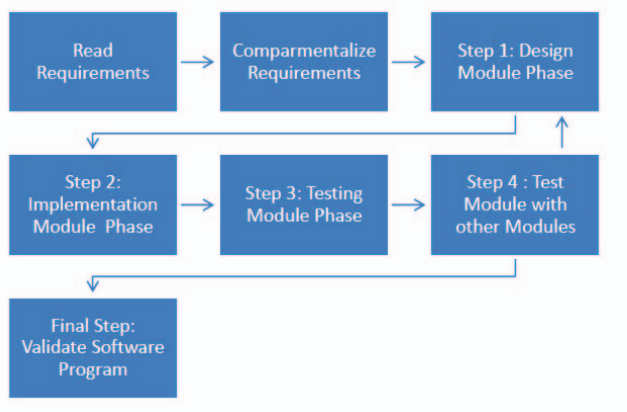

Fig. 5: Iterative process as defined by the final year cohort.

"However, I develop the software such that it's loosely coupled as much as possible so that if I were to make a mistake, I wouldn't need to change a lot of code. ",

while another notes how the use of regression testing coupled with a modular design leads to better code:

"I build regression tests for each component as it is created I tend to be able to break a project up into lots of small moving parts that are all regression tested whenever a change is made. This generally leads to fairly clean and bug free code."

Final year students also have a better understanding about how requirement analysis can be used to decompose a problem and design a good solution. A student notes:

"These kinds of use-scenarios sometimes uncover awkwardness in the program structure, or even missing features."

Other students include requirements analysis as their first, pre-design step, although the depth of their analysis is unclear:

"I normally start by reading the specification and identify the requirements of the software. Based on the specification and the specific requirements identified, a general design will be formulated"

D. Assessing Task Difficulty

Our analysis shows that the strategies employed in assessing task difficulty have also matured in final year students. In contrast to novice strategies, an increased number of students in the final year report on using prototyping and algorithm complexity to establish the difficulty of the assignment. A student notes:

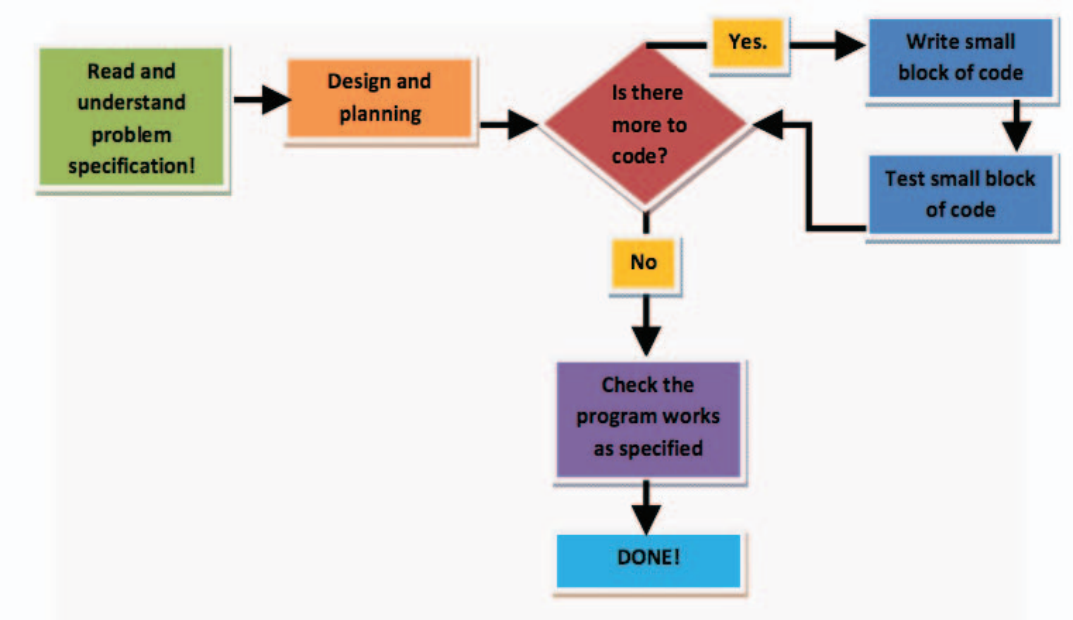

Fig. 4: Iterative process as defined by the novice cohort.

"Before implementing the actual assignment, I may choose to build a couple of small proof-of-concept exercises to test the difficult algorithms or language features mentioned above"

Other students report the extensive use of prototyping and test-driven-development to understand more complicated algorithms:

"If specific data analysis is required (bit shifting, binary operation etc.) I begin by running mock black box tests to understand what should be happening to data "

"Working on the algorithms first usually allows me to better judge the complexity of the system before I continue too far."

E. Time Management

It is important to note that time management strategies do not evolve substantially across cohorts, and, while final year students tend to prioritise tasks more, they do so on an "earliest-first deadline", that is, working on the assignments that are due first, and, within these assignments, working on the easiest or the less complex tasks first, before tackling difficult problems. Few students report working on core functional components that are key to developing a good solution. A student reflects on what (s)/he could do to improve:

"I believe I should minimise the time spent on dealing with little details and aim to tackle the core problems first and then subsequently deal with the little details later. This is because I often find myself, focusing too much on a single task and often delay the general implementation process of other core elements, which should arguably have been placed at a higher priority, and so by doing this, I would likely be able to construct a general solution a lot faster. "

VI. Discussion

Our analysis shows that CS SRLs evolve from novice to experts with respect to the use of SE-specific strategies involving design, the development of a software development process, and the integration of testing as a key point within this process. Our source coverage analysis supports these findings. Specifically, we observed that 22.4% of the novices developed design before coding. In contrast, 71.05% of the expert cohort developed design before coding. Similarly, the expert cohort is familiar with using design principles or standards to express their solutions, with 52.6% of the final year cohort reporting doing so, versus only 2.4% of the novices. Testing becomes fundamental to the software development process of the final year cohort, with 50% of the students reporting doing so. In contrast, only 12.9% of the novice students included comprehensive testing in their process. As the students mature into software engineers, their concerns about time management seem to take a secondary place, with the percentage of students reporting about time management dropping from 57.6% for novices to 26.8% for final year students. Our coverage analysis also shows that there are areas where students have yet to mature. Specifically, while there is a growth from 10% (novice) to 24% (final year students) of students reporting on allocating time for prototyping, this percentage is not as high as we would have wanted. Similarly, only 13.15% of the final year students report on validating their designs (up from 11.8%). We discuss ways of improving this in the below.

A. Improving strategies through reflection

Sharing of mental models and strategies is viewed as a powerful tool in cognitive development [18], with the use of students' own reflections and strategies providing a useful tool in helping instructors unpack and articulate their own processes in a way that is understandable and effective for learners. The development of mental models can be assisted through explicit sharing of models between students, and structuring of educational activities. Seel et al [33] identifies that mental models are not necessarily developed solely from instruction, but through restructuring and accommodating new knowledge within pre-existing causal relationships. There exists a natural resistance to assimilating new knowledge, and hence forming new mental models, which needs to be overcome by the student. These cognitive conflicts require significant effort by the student, and can be assisted by *guided discovery* teaching models [33], a form of cognitive apprenticeship [34], whereby the student is partially guided through the development of a problem statement, problem solving strategy and perhaps evaluation within an authentic context. The more guidance that is provided, the risk that the student will form incorrect mental models is reduced. In addition, resistance can be overcome through the introduction of exercises that explicitly challenge existing assumptions, such as the utility of design processes, or the continued use of known techniques [14].

B. Reflection over software development process

Final year students, while showing more sophisticated software development processes than their novice counterparts, can still improve, especially by validating their design early in the process and by the integration of software quality attributes in the design of their solution. These improvements can be achieved through a reflective process, in which students evaluate their past projects and derive lessons that can be applied in the future. These postmortems analyse past approaches both quantitatively and qualitatively, requiring the introduction of reflections and of measurements that students can easily access across the entire curriculum [35], starting with their first year of study. Moreover, in our curriculum, we introduce the discussion of quality attributes only in the final year. However, our analysis has shown that first year students, while lacking terminology, have a basic understanding of such attributes. This suggests that an earlier introduction of such discussion, perhaps through the use of open-ended problem solving and design exercises, might be beneficial.

C. Open-ended problem solving and design

To allow students to understand the importance of validating design, open-ended problem solving and design exercises could be used [36]. These exercises could also allow students to identify the core functional components in the design, and to plan their development accordingly. Moreover, such systems can also be used in teaching time-on-task estimation, a skill that our students are still missing. This could be done by discussing and adapting time estimation techniques such as those used in the teaching of agile development [37] or extreme programming [38], using user stories or planning games [39].

D. Early integration exercises

The majority of students in the final year cohort still elected to start their implementation with the least difficult task, with only one student reporting that he had employed a top-down integration strategy, by developing a stub-based interface design, and implementing each core component and testing the implementation as it advanced. We believe that there is a need for our students to become aware of various integration strategies early in their education. This could be achieved by effectively integrating testing in all the areas of the curriculum [40], but also through the introduction of integration and testing workshops in first year courses.

VII. Conclusion

In this paper, we present the evolution of discipline-specific self-regulated learning strategies across a Computer Science undergraduate curriculum, starting with a novice cohort approaching software design processes for the first substantial time, and a final year cohort in their final semester. Building on our earlier analysis of novice students and their approach to developed CS-specific self-regulated learning strategies, we extend this further with an analysis of the CS-specific strategies adopted by final year students (as proto-experts), exploring how the frequency of use of strategy has evolved, the evolution of strategies themselves as students gain more knowledge and experience in software development, and the introduction of new strategies designed to integrate new knowledge and skill into their learning processes.

We identify that as students evolve their strategies, they become more reliant on CS-specific strategies, and less reliant on the general SRL strategies that they adopted more commonly as novices. The final year cohort are more able to articulate the use of CS standards and processes in their design process, and place a greater importance on quality attributes and the development of critical components, indicating a maturity in process. In terms of general SRL strategies deemed to be successful, the final year cohort emphasised reflection over their process as a key strategy. While time management continues to be an issue, other concerns such as accessing assistance and seeking support no longer appear to be of concern.

While identifying increasing maturity and sophistication in the strategies employed by the final year cohort in terms of software development process, assessing task difficulty and decomposition and prioritisation, we were also clearly able to identify strategic areas lacking in the appropriate or expected level of sophistication, namely, design validation and prototyping as an aid to the design process. Our aims in understanding the evolution of software development processes across our curriculum have been to identify appropriate strategies that students find particularly relevant to their development as

Computer Scientists and Software Engineers, and to explore opportunities for increased scaffolding and reflection to assist strategy development, guided by those identified strategies. In our earlier analysis we identified pedagogical interventions designed to assist novices in their development of initial CS-specific strategies. In this discussion, we have explored the evolution of discipline-specific strategy further, and have identified examples of course and assessment scaffolding designed to assist in the development of these more advanced strategies, and - more specifically - the development of those strategies that we wish to encourage further.

REFERENCES

[1] J. Sheard, Simon, M. Hamilton, and J. Lönnberg, "Analysis of research into the teaching and learning of programming," in *Proceedings of ICER'09*, 2009, pp. 93–104.

[2] B. Zimmerman and M. M. Pons, "Development of a structured interview for assessing student use of self-regulated learning strategies," *American Educational Research Journal*, vol. 23, no. 4, pp. 614–628, 1986.

[3] G. Schraw, K. Crippen, and K. Hartley, "Promoting self-regulation in science education: Metacognition as part of a broader perspective on learning," *Research in Science Education*, vol. 36, no. 1-2, pp. 111–139, 2006.

[4] R. Cantwell and P. Moore, "The development of measures of individual differences in self-regulatory control and their relationship to academic performance," *Contemporary Educational Psychology*, vol. 21, pp. 500–517, 1996.

[5] P. Winne, "Inherent details in self-regulated learning," *Educational Psychologist*, vol. 80, pp. 284–290, 1995.

[6] C. M. Allwood, "Novices on the computer: A review of the literature," *Int. J. Man-Mach. Stud.*, vol. 25, no. 6, pp. 633–658, Dec. 1986. [Online]. Available: http://dx.doi.org/10.1016/S0020-7373(86)80079-7

[7] P. N. Robillard, "The role of knowledge in software development," *Communications of the ACM*, vol. 42, no. 1, January 1999.

[8] D. Schon, *The Reflective Practitioner: How Professionals Think in Action*. Basic Books, 1983.

[9] B. Hanks, L. Murphy, B. Simon, R. McCauley, and C. Zander, "Cs1 students speak: Advice for students by students," in *Proceedings of SIGCSE'09*, 2009, pp. 19–23.

[10] V. Ramalingam, D. LaBelle, and S. Wiedenbeck, "Self-efficacy and mental models in learning to program," in *Proceedings of ITiCSE'04*, 2004, pp. 171–175.

[11] K. Falkner, R. Vivian, and N. J. Falkner, "Identifying computer science self-regulated learning strategies," in *Proceedings of the 2014 Conference on Innovation & Technology in Computer Science Education*, ser. ITiCSE '14. New York, NY, USA: ACM, 2014, pp. 291–296. [Online]. Available: http://doi.acm.org/10.1145/2591708.2591715

[12] S. Paris and J. Turner, *Student Motivation, Cognition and Learning: Essays in Honor of Wilbert J. McKeachie*. Hillsdale, N.J.: Erlbaum, 1994, ch. Situated Motivation, pp. 213–237.

[13] E. Lichtinger and A. Kaplan, *Self-Regulated Learning*. Jossey-Bass, 2011, ch. Purpose of engagement in academic self-regulation, pp. 9–19.

[14] M. Veenman, J. Elshout, and J. Meijer, "The generality vs domain-specificity of metacognitive skills in novice learning across domains," *Learning and Instruction*, vol. 7, no. 2, pp. 187–209, 1997.

[15] J. Kramer, "Is abstraction the key to computing?" *Communications of the ACM*, vol. 50, pp. 36–42, April 2007.

[16] A. Schoenfeld and D. Hermann, "Problem perception and knowledge structure in expert and novice mathematical problem solvers," *Journal of Experimental Psychology: Learning, Memory and Cognition*, vol. 8, pp. 484–494, 1982.

[17] J. Bergin, M. Clancy, D. Slater, M. Goldweber, and D. B. Levine, "Day one of the objects-first first course: what to do," *SIGCSE Bull.*, vol. 39, no. 1, pp. 264–265, 2007.

[18] S. Violet, "Modelling and coaching of relevant metacognitive strategies for enhancing university students' learning," *Learning and Instruction*, vol. 1, pp. 319–336, 1991.

[19] S. Morra, C. Gobbo, Z. Marini, and R. Sheese, *Cognitive Development: Neo-Piagetian Perspectives*. Lawrence Erlbaum Associates, 2008.

[20] L. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, 1978.

[21] T. Caruso, N. Hill, T. VanDeGrift, and B. Simon, "Experience report: Getting novice programmers to think about improving their software development process," in *Proceedings of SIGCSE'11*, 2011, pp. 493–498.

[22] A. J. Dutson, R. H. Todd, S. P. Magleby, and C. D. Sorensen, "A review of literature on teaching engineering design through project-oriented capstone courses," *Journal of Engineering Education*, vol. 86, no. 1, pp. 17–28, 1997.

[23] C. Szabo, "Evaluating gamedevtycoon for teaching software engineering," in *Proceedings of the 45th ACM technical symposium on Computer science education*. ACM, 2014, pp. 403–408.

[24] W. S. Humphrey, *Introduction to the personal software process (sm)*. Addison-Wesley Professional, 1996.

[25] S. K. Lisack, "The personal software process in the classroom: Student reactions (an experience report)," in *Software Engineering Education & Training, 2000. Proceedings. 13th Conference on*. IEEE, 2000, pp. 169–175.

[26] P. M. Johnson and A. M. Disney, "The personal software process: A cautionary case study," *Software, IEEE*, vol. 15, no. 6, pp. 85–88, 1998.

[27] R. Bareiss, T. Sedano, and E. Katz, "Changes in transferable knowledge resulting from study in a graduate software engineering curriculum," in *Software Engineering Education and Training (CSEE&T), 2012 IEEE 25th Conference on*. IEEE, 2012, pp. 3–12.

[28] S. Crowe, K. Creswell, A. Robertson, G. Huby, A. Avery, and A. Sheikh, "The case study approach," *BMC Medical Research Methodology*, vol. 11, no. 1, 2011.

[29] J. Creswell, *Qualitative inquiry and research design: choosing among five approaches (3rd ed.)*. Thousand Oaks, California: SAGE Publications, 2012.

[30] M. Sandelowski, "Sample size in qualitative research," *Research in Nursing & Health*, vol. 18, no. 2, pp. 179–183, 1995.

[31] Y. Zhang and B. Wildemuth, *Application of social research methods to questions in information and library science*. Westport Conn: Libraries Unlimited, 2009.

[32] B. Zimmerman, "A social cognitive view of self-regulated academic learning," *Journal of Educational Psychology*, vol. 81, no. 3, pp. 329–339, 1989.

[33] N. Seel, S. Al-Diban, and P. Blumschein, *Integrated and Holistic Perspectives on Learning, Instruction and Technology*. Kluwer, 2000, ch. Mental Models & Instructional Planning, pp. 129–158.

[34] Q. Cutts, S. Esper, M. Fecho, S. Foster, and B. Simon, "The abstraction transition taxonomy: developing desired learning outcomes through the lens of situated cognition," in *Proceedings of ICER'12*, 2012, pp. 63–70.

[35] R. L. Upchurch and J. E. Sims-Knight, "Integrating software process in computer science curriculum," in *Frontiers in Education Conference, 1997. 27th Annual Conference. Teaching and Learning in an Era of Change. Proceedings.*, vol. 2. IEEE, 1997, pp. 867–871.

[36] E. Fernandez and D. Williamson, "Using project-based learning to teach object oriented application development," in *Proceedings of CITC'03)*, 2003, pp. 37–40.

[37] V. Devedzic and S. R. Milenkovic, "Teaching agile software development: A case study," *Education, IEEE Transactions on*, vol. 54, no. 2, pp. 273–278, 2011.

[38] A. Shukla and L. Williams, "Adapting extreme programming for a core software engineering course," in *Software Engineering Education and Training, 2002.(CSEE&T 2002). Proceedings. 15th Conference on*. IEEE, 2002, pp. 184–191.

[39] V. Mahnic, "A capstone course on agile software development using scrum," *Education, IEEE Transactions on*, vol. 55, no. 1, pp. 99–106, 2012.

[40] E. L. Jones, "Integrating testing into the curriculum — arsenic in small doses," in *Proceedings of the Thirty-second SIGCSE Technical Symposium on Computer Science Education*, ser. SIGCSE '01. New York, NY, USA: ACM, 2001, pp. 337–341. [Online]. Available: http://doi.acm.org/10.1145/364447.364617